

ELEC520

Python programming

Summer 2026

/list

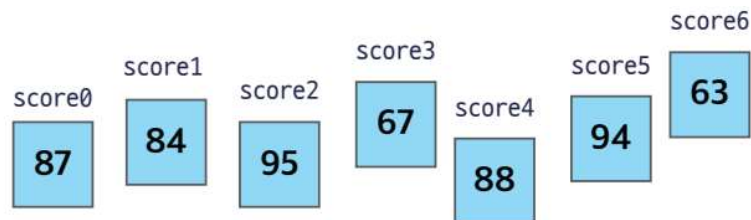
https://colab.research.google.com/drive/1wHQMDpPqa2GZd3E1EJaS_dXt_gtAwQL-?usp=sharing

리스트 자료형의 필요성



대화창 실습: 여러 개의 변수를 생성하고 사용하기

```
>>> score0 = 87
>>> score1 = 84
>>> score2 = 95
>>> score3 = 67
>>> score4 = 88
>>> score5 = 94
>>> score6 = 63
>>> print(score0, score3)
87 67
```



[그림 6-1] 개별적인 변수인 score0부터 score6으로 구성된 데이터

- 개별적인 변수의 생성과 값의 할당으로 이루어진 메모리 구조
- 많은 변수 이름이 필요할 것이다.
- 여러 곳의 메모리에 분산되어 저장되므로 연속적으로 읽을 수 없어 비효율적임
- 코딩이 복잡하고 에러의 가능성이 높아짐

리스트 **list**

- 개별적인 값을 하나의 변수에 담아서 처리
 - 매번 변수의 이름을 작성하고 관리하는 것보다 편리하고 효율적
 - 한꺼번에 복사하고 조작할 수 있다.
- 항목 **item** 또는 요소 **element**
 - 리스트를 이루는 원소로 심표로 구분된 자료 값



대화창 실습: 수치 값을 가진 리스트 만들기

```
>>> score_list = [87, 84, 95, 67, 88, 94, 63]
>>> score_list
[87, 84, 95, 67, 88, 94, 63]
>>> print(score_list[0], score_list[3])
87 67
```

요소의 참조는 리스트 이름과 인덱스를 사용한다

리스트의 메소드

method

append(), extend(), insert()

remove(), pop()

count(), index()

sort(), reverse()

리스트의 메소드

메소드	하는 일
<code>index(x)</code>	원소 <code>x</code> 를 이용하여 위치를 찾는 기능을 한다.
<code>append(x)</code>	원소 <code>x</code> 를 리스트의 끝에 추가한다.
<code>count(x)</code>	리스트 내에서 <code>x</code> 원소의 개수를 반환한다.
<code>extend([x1, x2])</code>	<code>[x1, x2]</code> 리스트를 리스트의 원소로 삽입한다.
<code>insert(index, x)</code>	원하는 <code>index</code> 위치에 <code>x</code> 를 추가한다.
<code>remove(x)</code>	<code>x</code> 원소를 리스트에서 삭제한다.
<code>pop(index)</code>	<code>index</code> 위치의 원소를 삭제한 후 반환한다. 이때 <code>index</code> 는 생략될 수 있으며, 이 경우 리스트의 마지막 원소를 삭제하고 이를 반환한다.
<code>sort()</code>	값을 오름차순 순서대로 정렬한다. <code>reverse</code> 인자의 값이 <code>True</code> 이면 내림차순으로 정렬한다.
<code>reverse()</code>	리스트를 원래 원소들의 역순으로 만들어 준다.

append(elt)

- 리스트의 마지막에 원소(elt) 추가

대화창 실습 : 리스트의 append() 메소드

```
>>> a_list = ['a', 'b', 'c', 'd', 'e']
>>> a_list.append('f')          # 'f' 항목 추가
>>> a_list
['a', 'b', 'c', 'd', 'e', 'f']
>>> n_list = [10, 20, 30, 40]
>>> n_list.append(50)           # 50 항목 추가
>>> n_list
[10, 20, 30, 40, 50]
```

extend(another_collection)

- 리스트에 다른 컬렉션 추가

대화창 실습 : 리스트의 extend() 메소드

```
>>> list1 = ['a', 'b', 'c']
```

```
>>> list2 = [1, 2, 3]
```

```
>>> list1.extend(list2) # extend()는 리스트 뒤에 리스트나 항목을 추가하는 메소드
```

```
>>> list1
```

```
['a', 'b', 'c', 1, 2, 3]
```

```
>>> list1.extend('d') # 인자로는 'd'와 같은 개별적인 원소 또한 삽입 가능
```

```
>>> list1
```

```
['a', 'b', 'c', 1, 2, 3, 'd']
```



LAB 6-5: 리스트 메소드의 응용

1. a 리스트의 원소 값이 [1, 2, 3]이고 b 리스트의 원소 값이 [10, 20, 30]이다. a 리스트에 `append(b)` 메소드와 `extend(b)` 메소드를 각각 호출할 때 어떤 결과가 나타날지 예측하시오. 그리고 실제로 수행한 후의 결과를 각각 적으시오.

```
>>> a = [1, 2, 3]
>>> b = [10, 20, 30]
>>> a.append(b)
>>> a
```

(1) _____

```
>>> a = [1, 2, 3]
>>> b = [10, 20, 30]
>>> a.extend(b)
>>> a
```

(2) _____

append() vs. extend()

만일 아래의 list1에 append()를 시도하게 되면 우리가 흔히 생각하는 [11, 22, 33, 44, 55, 66]의 결과가 나오지 않는다. list1.append([55, 66]) 메소드를 호출하면 [55, 66]이라는 리스트 항목을 [11, 22, 33, 44] 다음에 추가하여 list1은 [11, 22, 33, 44, [55, 66]]이라는 항목을 가지게 된다.

```
>>> list1 = [11, 22, 33, 44]
>>> list1.append([55, 66])
>>> list1
[11, 22, 33, 44, [55, 66]]
```

만일 리스트 내에 새로운 리스트의 항목을 추가하고자 할 경우에는 extend() 메소드를 사용할 수 있다. 다음과 같은 프로그램을 작성하여 수행해 보자.

```
>>> list1 = [11, 22, 33, 44]
>>> list1.extend([55, 66])
>>> list1
[11, 22, 33, 44, 55, 66]
```

insert(index, elt)

- 리스트의 특정 위치에 새로운 원소를 삽입

대화창 실습 : 리스트의 insert() 메소드

```
>>> list1 = ['a', 'c', 'd']
```

```
>>> list1.insert(1, 'b') # 1번째 위치에 'b' 삽입
```

```
>>> list1
```

```
['a', 'b', 'c', 'd']
```

remove(elt)

- 처음 나타나는 특정 원소(elt) 삭제

대화창 실습 : 리스트의 remove() 메소드

```
>>> list1 = ['a', 'b', 'c', 'b', 'd']
```

```
>>> list1.remove('b') # 제일 먼저 나타나는 'b' 원소를 삭제함
```

```
>>> list1
```

```
['a', 'c', 'b', 'd']
```

```
>>> list1.remove('b') # list1에 있는 'b' 원소를 삭제함
```

```
>>> list1
```

```
['a', 'c', 'd']
```

pop(index)

- 리스트의 특정 위치에 있는 항목을 삭제함과 동시에 이 항목을 **반환**
- 인자가 없을 경우 마지막 항목 삭제

```
>>> list1 = ['a', 'b', 'c', 'd']
>>> list1.pop()      # list1의 마지막 원소 'd'를 삭제하고 반환
'd'
>>> list1
['a', 'b', 'c']
>>> list1 = ['a', 'b', 'c', 'd']
>>> list1.pop(1)     # list1의 두 번째 원소 'b'를 삭제하고 반환
'b'
>>> list1
['a', 'c', 'd']
```

remove()를 이용한 마지막 원소의 삭제

- remove()는 삭제만 수행할 뿐 **삭제한 항목을 반환하지 않는다**

대화창 실습 : remove()와 인덱스를 이용한 마지막 원소의 삭제

```
>>> list1 = ['a', 'b', 'c', 'd']
>>> list1.remove(list1[-1])      # pop()과 유사하게 리스트의 마지막 원소 삭제
>>> list1
['a', 'b', 'c']
```

index(elt)

- elt의 인덱스 반환: elt가 여러 개 있다면 처음 인덱스

대화창 실습 : 원소의 인덱스를 반환하는 index() 메소드

```
>>> a_list = ['a', 'b', 'c', 'd', 'e']
```

```
>>> a_list.index('a') # 'a' 원소의 인덱스를 반환함
```

```
0
```

```
>>> a_list.index('b')
```

```
1
```

```
>>> a_list.index('x') # 존재하지 않는 'x' 원소의 인덱스를 조회 - 오류 발생
```

```
...
```

```
ValueError: 'x' is not in list
```

count(elt)

- 특정 원소 elt의 개수 반환

대화창 실습 : 리스트의 count() 메소드

```
>>> b_list = ['a', 'b', 'c', 'a', 'b', 'a']
```

```
>>> b_list.count('a') # 'a'가 몇 개 있는지 그 개수를 반환
```

```
3
```

```
>>> b_list.count('b') # 'b'의 개수를 반환
```

```
2
```

reverse()

- 리스트의 원소를 역순으로 **재배열**
- 내림차순 정렬을 하는 것은 아님

대화창 실습 : 리스트의 reverse() 메소드

```
>>> list1 = [100, 40, -20, 30]
>>> list1.reverse() # 리스트의 원소를 역순으로 다시 배열함
>>> list1
[30, -20, 40, 100]
```

lst[::-1]과의 차이는??

sort()

- 리스트를 오름차순으로 정렬
- 내림차순 정렬하려면 reverse인자를 True로!

sorted()와의 차이는??

대화창 실습 : 리스트와 sort() 메소드

```
>>> list1 = [20, 10, 40, 50, 30]
```

```
>>> list1.sort()           # list1의 원소를 오름차순 정렬
```

```
>>> list1
```

```
[10, 20, 30, 40, 50]
```

```
>>> list1.sort(reverse = True)  # list1의 원소를 내림차순 정렬
```

```
>>> list1
```

```
[50, 40, 30, 20, 10]
```

2. 1부터 10까지의 정수 값을 원소로 가지는 nlist를 생성하여 다음과 같이 출력하여라.

```
nlist = [1, 2, 3, 4, 5, 6, 7, 8, 9, 10]
```

3. insert() 메소드를 사용하여 nlist의 제일 앞에 0을 삽입하여 다음과 같이 출력하여라.

```
nlist = [0, 1, 2, 3, 4, 5, 6, 7, 8, 9, 10]
```

4. 위의 3번 문제를 통해 만들어진 nlist를 역순으로 다시 배열하여 다음과 같이 출력하여라 (reverse() 메소드 사용할 것).

```
마지막 원소 = 0  
nlist = [10, 9, 8, 7, 6, 5, 4, 3, 2, 1]
```

5. nlist의 제일 마지막 원소를 pop() 메소드를 사용하여 반환하여 출력하고 삭제한 후의 nlist를 다음과 같이 출력하여라.

```
마지막 원소 = 0  
nlist = [10, 9, 8, 7, 6, 5, 4, 3, 2, 1]
```